

차세대 크로스에셋 금융 데이터 시각화 대시보드 구축 및 바이브 코딩(Vibe Coding) 실행 마스터플랜

현대의 금융 데이터 분석 및 시각화 플랫폼은 방대한 양의 정량적 데이터와 정성적 인사이트를 동시에 처리하여 사용자에게 직관적인 판단의 근거를 제공해야 하는 복합적인 요구사항에 직면해 있다. 본 보고서는 다중 자산군(Cross-Asset)의 상승 확률 및 시계열 변곡점 궤적을 일반 투자자와 사용자에게 투명하게 제공하기 위한 맞춤형 웹 대시보드 시스템의 설계 및 구현 전략을 포괄적으로 다룬다. 특히, 인공지능 기반의 자동화된 개발 방법론인 바이브 코딩(Vibe Coding)을 활용하여 시스템을 구축한다는 전제하에, 프론트엔드 시각화 기법, 백엔드 데이터 처리 아키텍처, 스토리지 최적화 알고리즘, 그리고 초기 보안 모델에 이르기까지 AI 에이전트가 완벽하게 코드를 생성할 수 있도록 요구사항을 극도로 구체화하고 논리적으로 분석하였다.

시스템의 핵심 목적은 관리자가 복잡한 형태의 정량적 데이터 파일(CSV)과 정성적 보고서(PDF), 그리고 분석 이미지들을 최소한의 마찰로 업로드하고, 시스템이 이를 가공하여 모던하고 세련된 사용자 인터페이스(UI)를 통해 배포하는 것이다. 이를 위해 시스템은 세 가지 주력 모듈로 구성된다. 첫째, 9 개 주요 자산군에 대한 상승 확률 시계열 데이터 파싱 및 렌더링 모듈, 둘째, 다이내믹 시계열(DTW) 분석에 기반한 주간 단위 자산별 궤적 이미지 및 텍스트 매핑 모듈, 셋째, 자연어 처리 기반의 무작위 스니펫 추출 기능이 포함된 PDF 공지사항 모듈이다. 더불어 서버 스토리지의 한계를 극복하기 위한 자동 가비지 컬렉션(Garbage Collection) 시스템과 초기 프로토타이핑을 위한 하드코딩 기반 인증 시스템이 통합 구현되어야 한다. 이 모든 과정은 21st.dev 의 디자인 철학을 계승한 다크 모드 기반의 시각화 프레임워크 위에서 구동된다.¹

본 보고서는 이러한 복합적 요구사항을 바이브 코딩 엔진(예: Cursor, Windsurf 등)에 직접 입력하여 즉각적이고 오류 없는 코드를 산출해낼 수 있도록 고안된 아키텍처 명세서이자 개발 지침서로서 기능한다.

1. 시스템 아키텍처 및 디자인 시스템 철학

1.1 기술 스택 및 풀스택 프레임워크 선정

본 시스템은 관리자의 파일 업로드 및 데이터 파싱 과정과 사용자의 데이터 조회 과정이 하나의 애플리케이션 내에서 매끄럽게 이루어져야 한다. 따라서 프론트엔드와 백엔드가 긴밀하게 결합된 풀스택 프레임워크인 Next.js (App Router 기반)를 시스템의 중추로 채택한다.² Next.js 의 서버 컴포넌트(Server Components)와 서버 액션(Server Actions) 기능은 바이브 코딩 환경에서 외부 API 라우트를 별도로 구축하는 복잡성을 줄여주며, 관리자가 업로드하는 대용량 CSV 파일이나 PDF 문서를 브라우저가 아닌 서버 단에서 안전하고 신속하게 처리할 수 있는 환경을 제공한다.³

데이터 시각화를 위한 UI 프레임워크로는 Tailwind CSS 와 shadcn/ui 를 결합하여 사용한다.¹

shadcn/ui 는 컴포넌트를 패키지 형태로 설치하는 것이 아니라 소스 코드를 프로젝트 내부에 직접 복사하여 사용하는 방식을 취하므로, AI 에이전트가 요구사항에 맞추어 컴포넌트의 내부 로직이나 스타일을 즉각적으로 수정하고 확장하는 데 극대화된 유연성을 발휘한다.⁶ 상태 관리 및 데이터베이스 접근은 초기 프로토타이핑 단계의 기민성을 유지하기 위해 로컬 파일 시스템(JSON 기반 직렬화)을 활용하되, 추후 관계형 데이터베이스로의 전환을 염두에 두고 레포지토리(Repository) 패턴을 적용하여 데이터 계층을 추상화한다.

1.2 21st.dev 기반 모던 다크 모드 및 벤토 그리드(Bento Grid) 미학

금융 데이터를 다루는 대시보드의 디자인은 사용자의 인지적 부하를 최소화하면서도 방대한 정보를 한눈에 파악할 수 있도록 고도의 시각적 계층 구조를 갖추어야 한다. 이를 위해 21st.dev 에서 널리 활용되는 Bloomberg 터미널에서 영감을 받은 'Dark-First' 디자인 미학을 시스템 전반에 강제 적용한다.¹ 단순히 배경을 검은색으로 반전시키는 것을 넘어, 순흑색(#000000 또는 #09090b)의 바탕에 은은한 톤의 다크 그레이 카드 배경(#18181b), 그리고 데이터를 강조하기 위한 앰버(Amber) 또는 에메랄드(Emerald)와 같은 고대비 단색 액센트를 사용하여 전문가용 소프트웨어 특유의 밀도 높은 데이터 표현력을 확보한다.¹

레이아웃 측면에서는 최근 각광받고 있는 벤토 그리드(Bento Grid) 레이아웃을 도입한다.⁸ 벤토 그리드는 화면을 불규칙하지만 조화로운 사각형 컴포넌트들로 분할하여, 서로 다른 유형의 데이터(예: 거시적 확률 차트, 주간 궤적 이미지, 텍스트 요약 등)를 하나의 뷰포트 내에 효율적으로 융합하는 레이아웃 기법이다.⁸ 중앙의 가장 큰 그리드 영역에는 전체 자산군의 거시적 지표를 배치하고, 주변부의 작은 그리드에는 개별 자산의 텍스트 분석 내용이나 PDF 공지사항 스니펫을 배치함으로써 사용자는 스크롤을 최소화한 상태에서도 시장의 전반적인 맥락을 파악할 수 있다.

2. 모듈 1: 다중 자산군 상승 확률 시각화 시스템

시스템의 첫 번째 핵심 요구사항은 관리자가 0424_class_total.csv 파일을 업로드하고 기준일을 설정하면, 시스템이 9 개 자산군에 대한 상승 확률을 차트로 시각화하고, 관리자가 기입한 각 자산군별 설명을 함께 사용자에게 제공하는 기능이다. 이 모듈은 데이터의 파싱, 시각적 매핑, 그리고 텍스트 융합의 세 가지 단계로 구성된다.

2.1 CSV 데이터 구조 분석 및 서버 사이드 파싱 메커니즘

관리자가 업로드하는 0424_class_total.csv 파일은 절대적인 가격 궤적이 아니라 미래 50 주간의 '상승 확률(Probability of Rise)' 변화를 담고 있는 거시적 모멘텀 지표이다.¹¹ 데이터 구조를 면밀히 분석해보면, 최상단 헤더에는 9 개의 핵심 자산군(S&P500, NASDAQ, KOSPI, GOLD, DXY, USDKRW, USDJPY, 10YUSB, WTI)이 나열되어 있으며, 그 아래 두 번째 계층의 헤더로 각 자산군마다 A, B, C 세 개의 하위 열이 존재한다.¹¹ 첫 번째 열은 시계열을 나타내는 Date(날짜) 필드이다.

표 1. 업로드 데이터 (0424_class_total.csv) 구조 분석 모델

Date	S&P500 (A)	S&P500 (B)	S&P500 (C)	NASDAQ (A)	NASDAQ (B)	...	WTI (C)
2026.5.1	84.20%	89.50%	84.20%	98.10%	88.60%	...	65.30%
2026.5.8	86.70%	86.70%	83.10%	98.20%	88.30%	...	64.10%
2026.5.15	68.50%	70.20%	54.30%	83.70%	51.20%	...	66.50%

이 이중 계층(Double-layered) 구조의 CSV 를 프론트엔드 차트 라이브러리가 이해할 수 있는 평탄화된 JSON 배열로 변환하는 과정이 필수적이다. 바이브 코딩 시 **Next.js** 의 서버 액션 내부에서 **papaparse** 또는 **csv-parser** 모듈을 구동하여 데이터를 파싱해야 한다. 서버 측 로직은 CSV 의 첫 번째 행을 순회하며 빈 문자열을 이전 컬럼의 자산군 이름으로 병합(Merge)하여 식별자를 확립하고, 두 번째 행의 A, B, C 식별자와 결합하여 **SP500_A**, **SP500_B** 와 같은 고유 키(Key)를 생성하는 파서를 구현해야 한다. 또한 파싱 과정에서 모든 퍼센티지(%) 기호를 제거하고 부동소수점(Float) 숫자로 형변환하는 전처리 작업이 수반되어야 한다.

2.2 상승 확률 매핑 및 Recharts 시각화 연동

파싱된 데이터는 **shadcn/ui** 의 차트 컴포넌트를 통해 시각화된다. 이 컴포넌트는 내부적으로 **Recharts** 라이브러리를 사용하며, SVG 기반의 렌더링을 통해 다크 모드 환경에서도 유려한 애니메이션과 상호작용을 제공한다.¹² 요구사항에 따라 9 개 자산군에 대한 개별 차트가 렌더링되어야 하며, 각 차트는 A, B, C 세 개의 데이터 라인을 동시에 겹쳐 표시해야 한다.

차트의 Y 축(Y-Axis)은 가격 지수가 아닌 확률 지표이므로, 데이터의 최댓값에 관계없이 도메인을 0 에서 100(혹은 오버슈팅을 감안하여 120)으로 명시적으로 고정해야 한다.¹¹ 이는 사용자로 하여금 시장 체제(Regime)가 강한 강세장에 위치하는지, 혹은 침체 국면으로 진입하는지를 일관된 척도에서 판단할 수 있게 한다. 또한, 차트 영역에는 **shadcn/ui** 의 **ChartTooltip** 컴포넌트를 배치하여, 사용자가 특정 주차(Date)에 마우스를 올리면 A, B, C 세 모델의 확률값이 동시에 비교되는 인터랙티브 툴팁을 제공해야 한다.¹² 다크 테마와의 조화를 위해 차트 선의 색상은 채도가 높은 형광 블루, 퍼플, 그리고 에메랄드 색상 토큰을 CSS 변수로 할당하여 시인성을

극대화한다.¹

2.3 자산별 텍스트 기록 및 데이터 융합 인터페이스

관리자는 CSV 업로드와 기준일 설정에 그치지 않고, 9 개 자산군 각각에 대한 정성적인 코멘트(내용)를 기록할 수 있어야 한다. 이를 위해 관리자 대시보드의 업로드 인터페이스는 단순히 파일 선택 버튼 하나로 끝나는 것이 아니라, CSV 파일을 파싱한 직후 화면 상에 9 개의 텍스트 입력 영역(Textarea)을 동적으로 생성하는 워크플로우를 가져야 한다.

관리자가 각 자산에 대한 현황이나 예측 코멘트를 입력하고 최종 저장 버튼을 누르면, 시스템은 파싱된 CSV JSON 데이터, 기준일(Reference Date), 그리고 9 개의 텍스트 데이터 객체를 하나로 묶어 로컬 스토리지 또는 데이터베이스에 저장한다. 이후 일반 사용자 뷰에서는 벤토 그리드 레이아웃의 각 자산 카드 상단에 해당 차트가 렌더링되고, 하단 영역에 관리자가 작성한 텍스트가 스크롤 가능한 형태로 우아하게 배치된다. 이러한 구조는 차가운 정량적 데이터와 전문가의 따뜻한 정성적 통찰이 결합된 완벽한 금융 분석 보고서를 사용자에게 제공한다.

3. 모듈 2: 주간 다이내믹 시계열(DTW) 롤링 시프트 구현

두 번째 핵심 모듈은 0508_rolling_shift.pdf 문서에 설명된 멀티 타임프레임 크로스셋 변곡점 자료를 웹 상에서 구현하는 것이다. 이 자료는 주식, 달러, 환율, 금 등 9 개 자산이 향후 수개월간 어떤 궤적을 그릴지 예측하는 DTW(Dynamic Time Warping) 알고리즘의 결과물을 담고 있다.¹¹ 관리자는 매주 주간 단위로 이 궤적을 나타내는 이미지 파일들을 업로드하고, 간략한 텍스트 설명을 덧붙이는 작업을 수행해야 한다.

3.1 DTW 궤적 데이터의 구조적 이해 및 관리자 업로드 워크플로우

DTW 시계열 모델은 과거 30 년간의 방대한 금융 데이터 속에서 현재의 시장 모멘텀과 가장 유사한 과거 국면을 찾아내어 미래 궤적을 투사하는 기법이다.¹¹ 시각 자료에는 통상적으로 두꺼운 보라색 선(가장 유사한 5 번의 과거 평균을 의미하는 앙상블 평균선)과 주황색 점선(가장 똑같았던 단 한 번의 과거 사건을 나타내는 1 등 궤적 및 꼬리 위험 선)이 혼재되어 표시된다.¹¹ 투자자는 이 두 선의 괴리와 추세가 꺾이는 타이밍을 통해 자산군 이동(Shift)의 시그널을 얻는다.

표 2. DTW 궤적 매핑의 해석 기준 및 시각화 요소

시각화 요소	분석적 의미 (Financial Context)	시스템 적용 및 텍스트 작성 가이드
보라색 실선 (Ensemble	과거 5 개 유사 국면의 평균적 미래 궤적. 시장의 기본	관리자는 이 선의 상승/하락 기울기를 바탕으로 자산의

Average)	시나리오(Base Case)	중기적 추세 방향을 텍스트로 서술해야 함.
주황색 점선 (Rank 1 Trajectory)	역사상 가장 유사했던 단 하나의 과거 궤적. 극단적 발작이나 폭락의 꼬리 위험(Tail Risk) 테스트용	두 선의 궤적이 엇갈릴 때, 관리자는 시스템에 "변곡점 발생" 및 "자산 포트폴리오 스위칭" 경고를 기록함.
Y 축 지표	절대 가격이 아닌 추세와 에너지의 방향성을 나타내는 형태적 지표	시스템 UI 는 이미지를 왜곡 없이 원본 비율로 렌더링하여 형태적 기울기 판독을 보장해야 함.

시스템의 관리자 페이지는 이러한 분석론을 지원하기 위해 설계되어야 한다. 관리자는 주간 단위의 '새로운 롤링 시프트 생성' 버튼을 클릭한 후, 9 개 자산에 대한 이미지 업로드 슬롯이 나열된 폼을 마주하게 된다. 각 슬롯에는 차트 이미지 파일을 드래그 앤 드롭으로 업로드할 수 있는 영역과, 해당 차트에 담긴 변곡점의 의미를 서술하는 간략한 설명(Text Input) 필드가 존재한다. 서버 액션은 업로드된 9 개의 이미지 파일을 고유한 타임스탬프와 함께 정적 파일 서빙 폴더(/public/uploads/trajectories)로 복사하고, 이 경로와 텍스트 설명들을 하나의 세션 객체로 묶어 저장한다.

3.2 일반 사용자용 이미지 갤러리 및 모달(Modal) 인터랙션

일반 사용자가 접근하는 'Weekly Rolling Shift' 메뉴는 시각적인 몰입감을 극대화하는 갤러리 형태로 구성되어야 한다. 모던 다크 테마 환경에서 9 개의 자산 차트 이미지는 균일한 비율의 썸네일로 정렬된 그리드 뷰를 형성한다. 사용자가 특정 자산(예: WTI 원유)의 썸네일을 클릭하면, 브라우저 전체 화면을 어둡게 처리하는 오버레이(Overlay)와 함께 Shadow Dialog 혹은 Modal 컴포넌트가 활성화되며 고해상도의 차트 이미지가 화면 중앙에 부드럽게 렌더링된다.¹⁴

이미지 바로 하단에는 관리자가 입력한 '간략한 설명'이 우아한 타이포그래피로 표시된다. 이 UI 는 사용자가 여러 페이지를 이동할 필요 없이, 단일 뷰포트 내에서 팝업을 열고 닫으며 크로스에셋(주식, 외환, 채권 등) 간의 거대한 자금 이동 타이밍을 비교하고 직관적으로 이해할 수 있도록 돕는다.

4. 모듈 3: 공지사항 PDF 동적 파싱 및 무작위 스니펫 렌더링

시스템의 세 번째 기능적 기둥은 PDF 형식의 공지사항 관리이다. 관리자가 PDF 파일을 업로드하고 설명을 직접 작성하면 문제가 없으나, 바쁜 업무 환경으로 인해 설명 작성을 누락할 경우 시스템이 지능적으로 대응해야 한다. 시스템은 PDF 내용을 임의로 일부 추출하여 화면에 표시하는 자동화 로직을 수행하며, 이 기능이 기술적인 한계로 동작하지 않을 경우 관리자가 수동으로 파일 업로드와 설명을 병행하도록 하는 폴백(Fallback) 방식을 채택한다.

4.1 서버 사이드 PDF 텍스트 추출 메커니즘 (pdf-parse)

자바스크립트 생태계에서 PDF 를 다루는 방식은 크게 브라우저 단에서 렌더링을 돕는 `pdf.js` 와 서버 환경에서 텍스트 데이터만을 신속하게 추출하는 `pdf-parse` 등급의 라이브러리로 나뉜다.¹⁶ 클라이언트 브라우저에서 대용량 PDF 를 파싱하는 것은 메모리 누수와 메인 스레드 차단(UI Blocking)을 유발하여 다크 모드 대시보드의 유려한 애니메이션 성능을 저하시킬 수 있다.¹⁸ 따라서 본 시스템은 `Next.js` 의 서버 액션(Server Actions)을 활용하여 서버 측에서 가볍고 빠르게 텍스트를 추출하는 아키텍처를 적용한다.

관리자가 텍스트 설명 없이 PDF 를 업로드하면, 백엔드 라우트에서는 업로드된 버퍼(Buffer) 데이터를 `pdf-parse` 모듈로 전달한다. 이 라이브러리는 문서 내부에 내장된 문자열 정보를 순수 텍스트(Plain Text) 형태로 반환한다.¹⁷ 이 과정은 파일 I/O 작업이므로 비동기(async/await) 패턴으로 처리되어야 하며, 파싱 속도는 밀리초 단위로 이루어지므로 시스템 병목을 유발하지 않는다.

4.2 자연어 처리 기반 무작위 텍스트 스니펫 추출 알고리즘

서버 측에서 추출된 순수 텍스트는 불규칙한 개행과 불필요한 공백을 포함하고 있으므로 즉각적인 UI 렌더링에 적합하지 않다. 시스템은 추출된 원시 텍스트를 정제하고, 의미 있는 문장을 무작위로 발췌하는 알고리즘을 내부적으로 수행해야 한다.

- 텍스트 정제 및 분할:** 정규 표현식을 사용하여 다중 공백과 탭 문자를 단일 공백으로 치환한다. 이후 마침표(.), 물음표(?), 느낌표(!)와 같은 문장 종결 부호를 기준으로 전체 텍스트를 배열(Array of Sentences)로 분할한다.
- 데이터 필터링:** 분할된 문장 배열 중에서 목차, 단순 숫자, 특수 기호 등 무의미한 요소를 걸러내기 위해, 문자열 길이가 일정 기준(예: 30 자) 미만인 배열 요소는 필터링하여 제거한다.
- 무작위 발췌(Random Sampling):** 유효한 문장들로 구성된 최종 배열에서 자바스크립트의 `Math.random()` 함수를 활용하여 2 개에서 3 개 정도의 문장을 무작위로 인덱싱하여 추출한다.
- 조합 및 반환:** 추출된 문장들을 조합하여 하나의 요약된 스니펫(Snippet) 단락을 구성한다. 텍스트의 끝에는 줄임표(...)를 추가하여 이 문장이 문서의 일부임을 시각적으로 암시한다.

표 3. PDF 업로드 및 스니펫 생성 폴백(Fallback) 워크플로우

상황 시나리오	시스템 처리 로직 (System Logic)	프론트엔드 표출 결과 (UI Output)
Case 1: PDF 업로드 + 관리자 텍스트 입력 존재	PDF 파싱 생략. 파일은 스토리지에 저장하고, 입력된 텍스트를 DB 에 매핑	PDF 아이콘과 함께 관리자가 직접 작성한 텍스트 전문 표출
Case 2: PDF 업로드 + 텍스트 입력 누락	서버 측 pdf-parse 구동. 무작위 스니펫 추출 알고리즘 실행 및 결과 저장	PDF 아이콘과 함께 추출된 2~3 문장의 요약 스니펫과 [...] 표출
Case 3: PDF 추출 실패 (이미지 스캔본, 암호화 등)	파싱 라이브러리 예외(Exception) 발생. 빈 문자열 반환 감지	"설명 자동 추출에 실패했습니다." 안내 및 관리자 수동 입력 유도 UI 활성화

이러한 3 단계 폴백(Fallback) 시나리오는 시스템의 견고성(Robustness)을 극대화하며, 기술적 결함이 사용자 경험의 과편화로 이어지지 않도록 방어한다. 자동 추출이 불가능한 스캔 형태의 PDF 문서를 업로드하는 최악의 경우에도, 시스템은 다운타임 없이 관리자에게 수동 입력을 요청하는 우아한 저하(Graceful Degradation)를 달성한다.

5. 서버 인프라 최적화: 50 개 임계값 자동 가비지 컬렉션(GC)

웹 대시보드 시스템이 장기간 운영됨에 따라, 관리자가 업로드하는 CSV 데이터 파일, 고해상도의 주간 궤적 이미지 패키지(매주 9 개씩 생성), 그리고 공지사항 PDF 문서들은 서버의 로컬 스토리지 공간을 급격히 고갈시키는 원인이 된다. 시스템 요구사항은 외부 클라우드 스토리지(S3 등)의 의존 없이 단일 환경에서 스토리지 문제를 해결하기 위해 "각 기능별로 50 번 이상을 자동 삭제"하는 스토리지 관리 로직을 구축할 것을 명시하고 있다.

5.1 연대기적(Chronological) 스토리지 스캔 및 삭제 로직

이 기능은 각 모듈(분류 확률 CSV, 주간 채적 이미지 세트, 공지사항 PDF)에 데이터가 업로드되는 즉시 백그라운드 프로세스로 트리거되어야 한다. 데이터베이스 내의 레코드 숫자뿐만 아니라, 물리적인 파일 객체들까지 정확하게 삭제하여 좀비 파일이 남지 않도록 해야 한다. 구현의 핵심은 Node.js 의 기본 파일 시스템 모듈인 `fs.promises` 생태계를 활용하는 것이다.¹⁹

1. **목록화(Listing):** 특정 리소스의 업로드 디렉토리(예: `/public/uploads/pdf`)에 대해 `fs.readdir` 을 실행하여 폴더 내 모든 파일의 경로 배열을 확보한다.²⁰
2. **메타데이터 추출(Stat Extraction):** 배열 내의 각 파일에 대해 순회하며 `fs.stat` 명령을 비동기적으로 수행한다. 이 명령어는 파일의 물리적 생성 시간(`birthtime`) 및 최근 수정 시간(`mtime`) 메타데이터를 반환한다.¹⁹ 이를 기반으로 파일 이름과 시간 타임스탬프가 매핑된 객체 배열을 구축한다.
3. **내림차순 정렬(Sorting):** 구축된 배열을 최신 시간이 가장 앞단에 오도록 내림차순(`b.time - a.time`)으로 정렬한다.²² 이 정렬 과정을 거치면 배열의 첫 번째 요소부터 50 번째 요소(Index 0 ~ 49)까지는 보존해야 할 가장 최신 파일들이며, 51 번째 요소(Index 50)부터는 임계값을 초과한 구형 데이터로 정의된다.
4. **물리적 파기(Unlinking):** 임계값을 초과한 인덱스 영역을 잘라내어(`slice(50)`), 해당 파일들에 대해 `fs.unlink` 함수를 실행하여 디스크에서 영구적으로 삭제한다.¹⁹

5.2 동시성 제어 및 예외 처리

가비지 컬렉션 로직이 실행되는 동안 브라우저에서 동시에 여러 사용자가 파일을 읽고 있는 경우 충돌이 발생할 수 있다. 특히 Next.js 환경에서는 핫 리로딩(Hot Reloading) 메커니즘과 충돌하여 빌드 프로세스에 오류를 야기할 위험이 존재한다.²³ 이를 방지하기 위해 파일 삭제 루틴은 메인 렌더링 스레드를 차단하지 않는 비동기 이벤트 큐 안에서 실행되어야 하며, 특정 파일이 시스템에 의해 잠겨(Lock) 있어 unlink 작업에 실패하더라도 전체 스크립트가 크래시(Crash)되지 않도록 철저한 `try...catch` 블록으로 예외를 포획하고 로그로만 기록하는 유연성을 확보해야 한다.

6. 초기 보안 및 인증(Authentication) 모델

대부분의 현대적 시스템 설계에서는 NextAuth.js 나 Supabase Auth 와 같은 서드파티 인증 프로바이더를 연동하여 복잡한 세션 및 보안 로직을 위임하는 것이 관례이다. 하지만 본 프로젝트의 요구사항은 시스템의 기민한 초기 구현을 위해 "초기에는 인증 처리하지 않고 하드코딩된 id: root, pw: 1234 로 구현하고, 나중에 인증하는 것으로 변경"할 것을 요구하고 있다. 이 단순한 요구사항은 오히려 향후의 확장성을 해치지 않도록 아키텍처를 교묘하게 구성해야 함을 시사한다.

6.1 미들웨어(Middleware) 기반의 접근 제어

인증 시스템의 핵심은 코어 컴포넌트들을 수정하지 않고 보안 레이어를 독립시키는 것이다. Next.js 의 `middleware.ts` 파일을 활용하면 모든 HTTP 요청이 개별 페이지에 닿기 전에 사전

검열을 수행할 수 있다. 미들웨어는 클라이언트의 요청 경로(URL Path)를 분석하여, 해당 요청이 `/admin` 으로 시작하는 관리자 전용 경로인지 확인한다.

관리자 경로에 접근하려 할 때, 시스템은 브라우저 쿠키(Cookies) 딕셔너리에 `auth_session` 과 같은 검증 토큰이 존재하는지 조회한다. 만약 토큰이 없다면 사용자를 강제로 `/login` 페이지로 리다이렉트 시킨다. 로그인 페이지에서는 매우 단순한 서버 액션 함수가 동작한다. 클라이언트 폼에서 전달받은 아이디와 비밀번호 문자열이 정확히 `root` 와 `1234` 인지 평문으로 비교하고, 일치할 경우 만료 기한(Expiration)이 설정된 암호화되지 않은 쿠키를 브라우저에 구워 발급한다.

이러한 미들웨어 기반의 차단 로직은 향후 요구사항이 변경되어 고도화된 OAuth 인증을 도입해야 할 시기가 도래했을 때, 개별 컴포넌트들의 코드를 뜯어고칠 필요 없이 오직 미들웨어의 쿠키 검증 로직 한 줄만 JWT(JSON Web Token) 검증 로직으로 교체하면 되는 무한한 확장성을 제공한다.²⁴ 임시 조치임에도 불구하고 소프트웨어 공학의 개방-폐쇄 원칙(OCF)을 완벽하게 준수하는 설계이다.

7. 바이브 코딩(Vibe Coding)을 위한 시스템 프롬프팅 지침

본 대시보드의 구현은 개발자가 키보드를 타이핑하여 코드를 직조하는 방식이 아닌, AI 에이전트(Cursor, Windsurf 등)에게 의도를 전달하여 시스템을 자동 생성하는 바이브 코딩(Vibe Coding) 방법론을 통해 진행된다.²⁵ AI 에이전트는 환각(Hallucination)을 일으키거나 파일 간의 연결성을 잃어버리는 경향이 있으므로, 프롬프트는 단순한 요구사항의 나열이 아니라 엄격한 기술적 제약 조건과 단계별 실행 지침(Step-by-step Plan)을 포함한 "마스터 프롬프트" 형태로 주입되어야 한다.²⁷

아래의 논리적 구문들을 바이브 코딩 도구의 프롬프트 창에 순차적으로 입력함으로써 요구사항이 완벽하게 코드로 번역될 수 있다.

7.1 단계별 AI 에이전트 마스터 프롬프트 (Execution Prompts)

[Phase 1: 인프라 및 다크 모드 벤토 그리드 UI 초기화]

"너는 세계 최고 수준의 Next.js 및 Tailwind CSS 프론트엔드 아키텍트이다. 지금부터 21st.dev 미학을 따르는 금융 데이터 분석 대시보드를 구축한다. `npx create-next-app@latest` 를 사용해 App Router 기반 프로젝트를 생성하라. 즉시 `shadcn/ui` 를 초기화하고 모든 테마를 다크 모드(배경색 `#09090b`)로 강제 고정하라. 메인 대시보드 페이지(`app/page.tsx`)는 Bento Grid 레이아웃을 채택하여 상단에는 큰 요약 카드, 하단에는 다중 그리드 구조의 카드가 배치될 빈 틀을 구성하라. 모든 작업은 TypeScript 로 이루어지며 컴포넌트는 `src/components/ui` 에 저장하라. 여기까지 완료하면 결과를 보고하고 대기하라."

[Phase 2: 하드코딩 인증 및 파일 가비지 컬렉션 로직]

"이제 보안과 스토리지 코어 로직을 작성한다. 첫째, Next.js 의 `middleware.ts` 를 작성하여 `/admin` 하위의 모든 라우트를 보호하라. `app/login/page.tsx` 에 로그인 폼을 만들고, 제출된 아이디가 `root`, 비밀번호가 `1234` 인 경우 서버 액션에서 세션 쿠키를 생성해 부여하라. 둘째,

src/lib/storage.ts 에 cleanupOldFiles(dirPath, limit = 50) 함수를 작성하라. Node.js 의 fs.promises.readdir 과 stat 을 사용해 디렉토리 내 파일들의 수정 시간(mtime)을 읽고 내림차순 정렬한 뒤, 인덱스 50 을 초과하는 가장 오래된 파일들을 fs.promises.unlink 로 자동 삭제하는 로직을 완벽한 예외 처리(try/catch)와 함께 구현하라."

"관리자가 /admin/upload-csv 에서 0424_class_total.csv 파일을 업로드하고 '기준일(Date)'을 설정하는 폼을 만들어라. 업로드 폼 하단에는 S&P500, NASDAQ, KOSPI 등 9 개 자산군 각각에 대한 텍스트 설명을 적는 9 개의 텍스트 에어리어가 동적으로 있어야 한다. 제출 시 서버 액션 내에서 papaparse 를 사용해 CSV 를 파싱하고 JSON 으로 변환해 저장하라. 일반 사용자가 보는 메인 대시보드에서는 shadcn/ui 차트(Recharts)를 사용해 이 9 개 자산의 차트를 렌더링하라. Y 축 도메인은 ``으로 고정하여 확률의 높낮이를 명확히 하고, A, B, C 세 열의 데이터를 각각 다른 색상의 선으로 렌더링하라. 각 차트 하단에는 관리자가 적은 해당 자산군의 텍스트 설명이 표출되어야 한다. 데이터 갱신 시 Phase 2 에서 만든 50 개 삭제 로직을 호출하라."

"주간 타임프레임 변곡점을 추적하는 기능을 만든다. /admin/weekly-shift 경로에 9 개의 자산 차트 이미지 파일(png/jpg)을 동시에 업로드할 수 있는 폼을 구성하라. 각 이미지 옆에는 관리자가 '간략한 설명'을 적을 수 있는 입력칸이 묶여 있어야 한다. 일반 사용자는 이 데이터를 '주간 궤적 갤러리' 뷰에서 본다. 9 개의 썸네일 이미지가 그리드로 렌더링되며, 클릭 시 Shadcn Dialog(Modal)가 떴서 원본 이미지가 크게 보이고 그 하단에 관리자가 작성한 텍스트 설명이 렌더링되는 모던한 인터랙션을 구현하라. 저장 시 50 개 이미지 세트 초과 분은 삭제 로직과 연동하라."

"마지막으로 공지사항 모듈이다. /admin/notice 에서 PDF 문서와 텍스트 설명을 업로드 받는다. 관리자가 텍스트 설명을 비워둔 채 PDF 만 업로드할 경우, 서버 액션 안에서 pdf-parse 라이브러리를 사용해 PDF 의 순수 텍스트를 파싱하라. 파싱된 문자열을 정규식으로 문장 단위로 분할하고, 30 자 이상의 문장 중 2~3 개를 무작위(Random)로 추출하여 요약 스니펫 문장으로 합친 뒤 저장하라. 만약 pdf-parse 가 에러를 뱉거나 결과가 없다면, '내용을 추출할 수 없습니다'라고 처리하라. 사용자 뷰에서는 PDF 아이콘과 함께 이 랜덤 스니펫 또는 관리자 수동 텍스트가 표시되게 하라. pdf-parse 사용 시 빌드 오류를 막기 위해 next.config.js 의 serverComponentsExternalPackages 옵션을 적절히 조치하라."

8. 결론

본 문서는 단순히 시각화 차트를 그리는 웹사이트 제작을 넘어, 정교한 수학적 통계 모델(상승 확률 및 DTW 시계열 변곡점)의 결과를 일반 대중에게 가장 효과적으로 전달하기 위한 인지 과학적이고 소프트웨어 공학적인 설계도이다.

시스템은 Next.js 의 서버 액션을 적극 도입하여 클라이언트 브라우저의 부하를 근본적으로 차단함으로써, PDF 텍스트 파싱이나 대용량 CSV 데이터 처리에서 발생하는 전통적인 병목 현상을

해결하였다. 또한 다크 모드 특유의 눈부심 방지와 높은 명도 대비를 활용한 **21st.dev** 스타일의 벤토 그리드 레이아웃은, 방대한 9 개 자산군의 변동성을 사용자에게 압도감 없이 전달하는 최적의 그릇이 될 것이다.

스토리지 공간 고갈을 막기 위한 50 개 임계값 기반의 가비지 컬렉션 엔진과, 초기 프로토타이핑을 위해 치밀하게 분리된 미들웨어 인증 아키텍처는 시스템이 파일 쓰레기로 인해 다운되거나 추후 확장이 가로막히는 사태를 미연에 방지한다. 본 보고서의 제 7 장에 서술된 단계별 마스터 프롬프트 지침을 바이트 코딩 에이전트에 엄격하게 주입함으로써, 이 모든 복잡한 기술적 요구사항은 최소한의 수동 코딩 타이핑만으로, 가장 모던하고 견고한 형태의 크로스셋 금융 대시보드로 탄생할 수 있을 것이다.